

Métodos de boosting

A thick, hand-drawn style orange line that spans the width of the title text.

Jhon Quiza



Intuición

- Árboles de decisión → predictores de alta varianza.
- Métodos de bagging (Random Forest) → promedian para bajar varianza.
- El sesgo todavía puede ser alto.
- Los métodos de boosting reducen el sesgo combinando muchos estimadores débiles (weak learners) **secuencialmente**, cada uno corrigiendo los errores del anterior.

AdaBoost

Tema	Puntos clave
Origen	Freund & Schapire, 1996 (premio Gödel)
Algoritmo	Pondera instancias → entrena árbol débil → ajusta pesos de las instancias mal clasificadas
Fórmula final	Votación ponderada
Relación con pérdida	Minimiza la pérdida exponencial ; se interpreta como <i>gradient descent</i> en modelos aditivos.
Ventajas	Poco tuning, buen desempeño en tabular pequeño-medio
Limitaciones	Sensible a ruido y clases desbalanceadas; no soporta nulo/missing

Algoritmo AdaBoost

Primera
predicción

Calcula el error

Calcula el peso
del estimador en
la predicción final
→ α

Calcula el peso
de los errores en
el siguiente
estimador → w_i

Entrena un nuevo
estimador con los
pesos ajustados

Repite pasos 2, 3,
4 y 5

Combina las
predicciones

Algoritmo AdaBoost – Cálculo del error

Clasificadores:

$$\epsilon_t = \frac{\sum_{i=1}^N w_i * I(y_i \neq h_t(x_i))}{\sum_{i=1}^N w_i}$$

Donde:

- w_i es el peso de la instancia i , el cual es 1 en la primera iteración
- $I(y_i \neq h_t(x_i))$ es una función indicadora que vale 1 si la instancia fue mal clasificada y 0 si fue correctamente clasificada
- $h_t(x_i)$ es la predicción del clasificador t

Regresores:

- Calcula los residuos absolutos: $r_i = |y_i - \hat{y}_1(x_i)|$

Algoritmo AdaBoost

3. Cálculo del parámetro α :

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$$

Si el clasificador tiene un bajo error (ϵ_t) α_t será grande, lo que significa que este clasificador tendrá más peso en la predicción final.

4. Ajuste de pesos de las instancias:

$$w_i^{(t+1)} = w_i^t * \exp(-\alpha_t * y_i * h_t(x_i))$$

- Si una instancia es mal clasificada $y_i \neq h_t(x_i)$, su peso aumenta.
- Si una instancia es correctamente clasificada $y_i = h_t(x_i)$, su peso disminuye.

Algoritmo AdaBoost

4. **Entrenamiento de nuevos clasificadores:** Un nuevo clasificador débil se entrena en los datos con los pesos ajustados. El proceso de entrenamiento se repite iterativamente, hasta alcanzar un número predeterminado de iteraciones, o una tolerancia en el error.
5. **Combinación de los clasificadores:** Los clasificadores débiles entrenados se combinan mediante un voto ponderado, donde los clasificadores con menor error tienen más influencia en la decisión final, de acuerdo con esta fórmula:

$$\hat{y} = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$$

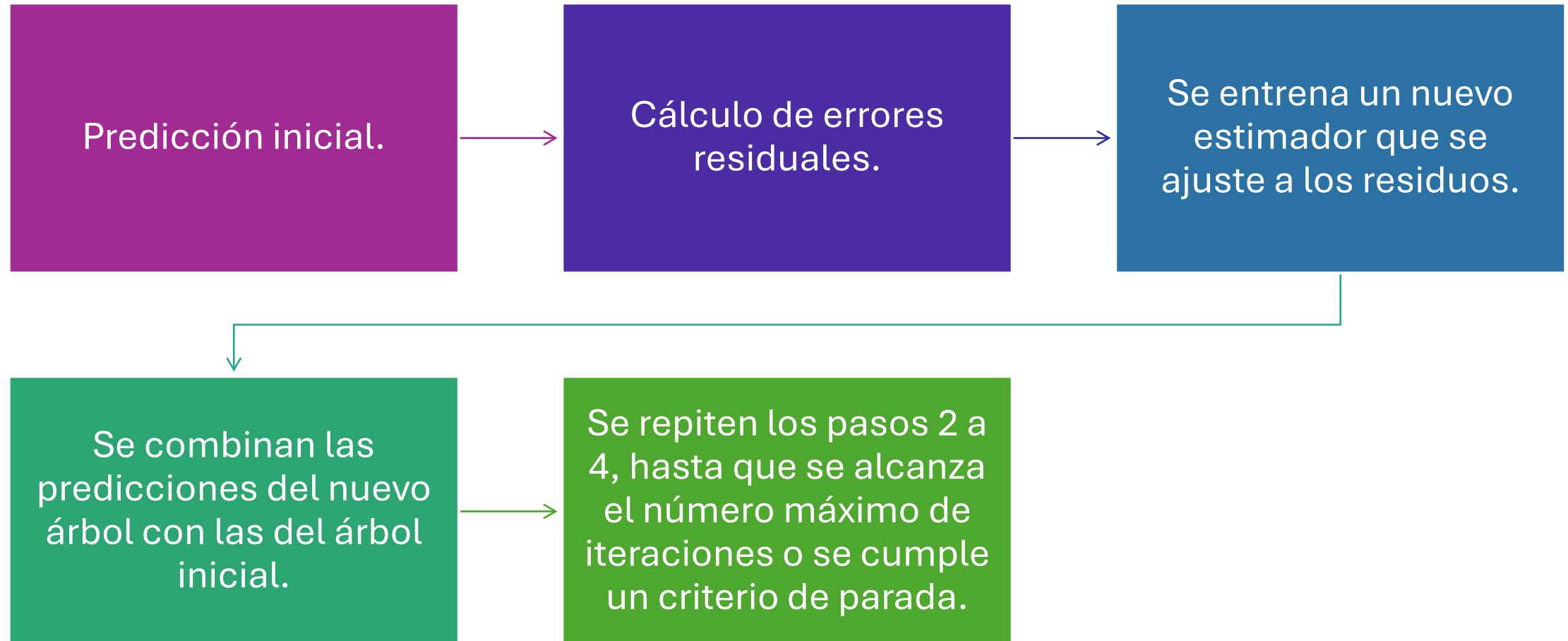
Donde:

- $h_t(x)$ es la predicción del clasificador t
- α_t es el peso asignado al clasificador t
- T es el número total de clasificadores (iteraciones)

Gradient Boosting

A thick, hand-drawn style orange line that underlines the title 'Gradient Boosting'.

Algoritmo Gradient Boosting



Algoritmo clasificador Gradient Boosting

1. **Inicialización:** Se empieza con una predicción inicial $F_0(x)$ que, en el caso de clasificación binaria, es la probabilidad promedio de pertenecer a la clase positiva en todo el conjunto de datos:

$$F_0(x) = \log \left(\frac{P(y = 1)}{P(y = 0)} \right)$$

En clasificación binaria, normalmente se utiliza el logaritmo de las probabilidades de clase, ya que es más fácil interpretar y optimizar.

Algoritmo de Gradient Boosting

2. **Cálculo de los residuos:** Los residuos representan las diferencias entre las predicciones actuales y los valores reales. El residuo en la iteración m para una observación i se calcula como:

$$r_i^{(m)} = -\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}$$

Donde:

- $L(y_i, F(x_i))$ es la **función de pérdida** (log-loss en clasificación):
$$L(y_i, \hat{y}_i) = -[y_i \cdot \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$
- y_i es la etiqueta real de la instancia i ,
- $F(x_i)$ es la predicción del modelo para la instancia x_i .

Algoritmo de Gradient Boosting

Para clasificación binaria, los residuos son el gradiente de la función de pérdida logarítmica respecto a la predicción del modelo, que es simplemente la diferencia entre las predicciones actuales y las etiquetas reales:

$$r_i^{(m)} = y_i - \hat{y}_i^{(m-1)}$$

Donde:

- y_i es la etiqueta real (0 o 1),
- $\hat{y}_i^{(m-1)}$ es la probabilidad de predicción del modelo en la iteración anterior.

Algoritmo de Gradient Boosting

3. Ajustar un nuevo modelo a los residuos: Se entrena un árbol para **predecir estos residuos**. El objetivo aquí es corregir el error residual cometido por el modelo anterior.
4. Actualización de las predicciones: El modelo final se actualiza sumando la predicción del nuevo árbol, multiplicada por un parámetro de tasa de aprendizaje. La predicción final se calcula como:

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

Donde:

- $F_m(x)$ es la predicción del modelo final en la iteración m ,
- $F_{m-1}(x)$ es la predicción del modelo en la iteración anterior,
- $h_m(x)$ es la predicción del nuevo árbol entrenado en los errores,
- η es el parámetro de tasa de aprendizaje (learning rate).

Algoritmo de Gradient Boosting

5. Repetir el proceso de ajustar un nuevo clasificador a los errores del modelo anterior se repite hasta que:
- Se alcanza un número máximo de iteraciones (T).
 - O el error no mejora significativamente.

Al final del proceso, la predicción del ensamble es la suma ponderada de todos los clasificadores débiles entrenados:

$$F_M(x) = F_0(x) + \eta h_1(x) + \eta h_2(x) + \dots + \eta h_M(x)$$
$$\hat{y}(x) = \sigma(F_M(x)) = \frac{1}{1 + e^{-F_M(x)}}$$

Regresor Gradient Boosting

1. Predicción inicial: por defecto la media del target, $F_0(x) = \bar{y}$

2. En cada iteración m :

- Calcula residuos como gradiente de la pérdida:

$$r_i^{(m)} = -\frac{\partial L(y_i, F_{m-1}(x_i))}{\partial F_{m-1}(x_i)}$$

- Para MSE: $r_i = y_i - F_{m-1}(x_i)$
- Ajusta un regresor $h_m(x)$ a esos residuos
- Actualiza: $F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$

3. Predicción final: $\hat{y}(x) = F_0(x) + \eta h_1(x) + \eta h_2(x) + \dots + \eta h_M(x)$

Vamos al código ...